P.Vishwa Nandan(192473015)

Q3 – Engineer the hashCode–equals Contract

Marks: 6

This question is about hashCode(), equals() and the Java HashSet collection.

Problem Context – Event Attendance De-duplication

Imagine a college fest where students scan their ID cards at different entry gates.

A student may scan their card multiple times. We need to ensure that the attendance system counts each student only once.

For example:

- Arun scans at Gate 1.
- Divya scans at Gate 2.
- Arun scans again at Gate 3.

The expected unique student count is **2**, not 3.

We use HashSet<Student> to store unique students.

---

1. Understanding equals() and hashCode()

Before writing the program, understand these two methods.

A. equals()

The equals() method compares two objects to determine whether they are logically equal.

Example:

Student s1 = new Student("192311001", "Arun");

Student s2 = new Student("192311001", "Arun");

System.out.println(s1.equals(s2));

Output:

true

Both students have the same registration number, so they are considered equal.

B. hashCode()

The hashCode() method returns an integer value representing an object for hash-based collections.

Example:

```
@Override
public int hashCode() {
   return regNo.hashCode();
}
```

Here, the registration number is used to generate the hash code.

Important Contract

Java follows this important rule:

If two objects are equal according to equals(), they must have the same hashCode().

However, two objects having the same hash code does not necessarily mean they are equal.

---

2. How HashSet Uses equals() and hashCode()

HashSet internally uses hashing to store and find objects.

The process works as follows:

1. HashSet calls hashCode() to determine the bucket where an object should be stored.
2. When another object is inserted into the same bucket, HashSet uses equals() to check whether it is a duplicate.
3. If the objects are equal, the duplicate is not inserted.
4. Otherwise, the new object is stored.

What happens if hashCode() is not overridden?

Java uses the default hashCode() implementation inherited from the Object class.

Two different Student objects with the same registration number may receive different hash codes.

Therefore, HashSet may place them in different buckets and fail to identify them as duplicates.

This violates the equals-hashCode contract.

---

3. Design Three Experiments

The assignment requires three experiments.

Experiment 1 – Same Object Scanned Twice

Suppose the same Student object is added twice.

Student s1 = new Student("192311001", "Arun");

present.add(s1);

present.add(s1);

Prediction:

The HashSet size will be 1 because the same object reference is added twice.

Experiment 2 – Different Objects with the Same Registration Number

Consider:

present.add(new Student("192311001", "Arun"));

present.add(new Student("192311001", "Arun"));

These are two separate objects with the same registration number.

Prediction before fixing hashCode():

The HashSet will likely store both objects because their default identity-based hash codes are usually different.

Expected typical result: size 2 for these two objects.

Prediction after fixing hashCode():

The HashSet will recognise both objects as equal and store only one.

Expected result: size 1.

Experiment 3 – Changing Registration Number After Insertion

Consider adding a student to HashSet and then changing the registration number.

If the registration number is used in hashCode() and gets modified after insertion, the object's hash code changes.

HashSet may no longer find the object in the bucket where it was originally stored.

Prediction:

The contains() method may return false even though the object is still present in the HashSet.

This demonstrates why fields used in equals() and hashCode() should not be modified while an object is stored in a HashSet.

Explanation

Initially, the registration number is A, so the object is stored in the corresponding hash bucket.

After changing it to B, the hash code also changes.

When contains() searches for the object, it looks in the new hash bucket instead of the original bucket.

Therefore, it returns false.

However, the object is still present in the HashSet.

Conclusion: Fields used in equals() and hashCode() should be immutable while objects are stored in hash-based collections.

This is why the corrected Student class uses final for regNo.

---

7. Experimental Results and Comparison

The following table summarises the three experiments.

Experiment	Before correction	After correction
Same object added twice	1	1
Separate objects with same registration number	Usually 2	1
Change registration number after insertion	Not applicable to final-field Student	Mutable test may fail contains()

For the exact assignment program:

Execution	Expected output
Before TODO 1	Typically Unique students: 3
After TODO 1	Unique students: 2

The correction ensures that the HashSet behaves according to the intended equality rules.

---

8. Alternative Concept and Complexity Analysis

The assignment asks us to compare HashSet with TreeSet and HashMap.

A. TreeSet with Comparator

TreeSet stores elements in sorted order.

We can use a Comparator based on registration numbers.

Example:

```
TreeSet<Student> students = new TreeSet<>(
   (s1, s2) -> s1.getRegNo().compareTo(s2.getRegNo())
);
```

For this approach, we would need to provide a getter for regNo.

TreeSet compares registration numbers to identify duplicates.

If two students have the same registration number, the second one is not inserted.

Complexity:

- Insertion: O(log n).
- Searching: O(log n).
- Space: O(n).

B. HashMap Using Registration Number

HashMap can store registration numbers as keys and Student objects as values.

Example:

```
HashMap<String, Student> students = new HashMap<>();

students.putIfAbsent(
   "192311001",
   new Student("192311001", "Arun")
);

students.putIfAbsent(
   "192311002",
   new Student("192311002", "Divya")
);
```

```
students.putIfAbsent(
    "192311001",
    new Student("192311001", "Arun")
);


System.out.println("Unique students: " + students.size());
```

Output:

Unique students: 2

Here:

- Key = Registration number.
- Value = Student object.

The putIfAbsent() method prevents an existing registration number from being replaced.

C. Complexity Comparison

Data Structure	Insertion	Searching	Space
HashSet	O(1) average	O(1) average	O(n)
TreeSet	O(log n)	O(log n)	O(n)
HashMap	O(1) average	O(1) average	O(n)

Here, n represents the number of unique students.

HashSet: Suitable when we need to store unique Student objects.

TreeSet: Suitable when unique students must also be maintained in sorted order.

HashMap: Suitable when registration numbers are used as keys to retrieve student details quickly.

Recommendation

HashSet is suitable for event attendance de-duplication because it stores unique student objects and provides efficient average-case insertion and searching.

However, HashMap is also a good option when registration numbers are the primary identifiers.

---

9. Conclusion

The hashCode() and equals() methods must follow a proper contract for HashSet to work correctly. When two students have the same registration number, they must produce the same hash code and be considered equal. Overriding hashCode() prevents duplicate attendance entries. TreeSet and HashMap are alternative solutions depending on whether sorted storage or key-based lookup is required.